



**FIREBRAND**

LYCEUM EDUCATION GROUP

# C# Fundamentals

C# Pathway | Session 1

# Session Recording & Professional Expectations

## Session Recording

- All Estio training sessions are recorded to:
  - Support learner progress
  - Enable review of content
  - Maintain quality and compliance standards
- By attending this session, you acknowledge that recording is taking place.

## Camera & Participation Expectations

- To support engagement and attendance monitoring:
  - Cameras should remain on during the session
  - Please participate actively in discussions and activities
  - If you are unable to use your camera or microphone for any reason, please inform your tutor directly via chat or message.

**This session forms part of your professional apprenticeship experience. Please engage respectfully and professionally throughout.**

# Safeguarding, British Values & Wider Learning

Safeguarding protects individuals from harm, abuse, neglect, radicalisation and exploitation.

If you have a concern about yourself or someone else, email:

[safeguarding@bpp.com](mailto:safeguarding@bpp.com)

Or speak to your tutor privately.

We treat all concerns seriously and confidentially.

## Safeguarding



The British Values are:

- 1. Democracy**
- 2. Rule of Law**
- 3. Individual Liberty**
- 4. Mutual Respect**
- 5. Tolerance of Different Faiths and Beliefs**

These values underpin everything we do in professional and digital environments.

## British Values



Prevent is about protecting people from being drawn into extremism or radicalisation.

In our programmes, Prevent links to:

- **Online safety**
- **Digital responsibility**
- **Critical thinking**
- **Professional ethics**

## Prevent



**As we go through today's session, consider:**

How today's topic connects to British Values

Whether there are any Prevent or digital safety themes

How this learning supports your professional and career development

Your tutor will guide discussion and help you make these connections.

# Competencies and Coverage

## Professional Discussion underpinned by Portfolio

### Knowledge

**K1**, K3, K4, K5, K7, K8,  
K10, K12

### Skills

S2, S3, S5, S8, S9, S13,  
S14, S15, S17

### Behaviours

B1, B4, B5, B6, B7, B8,  
B9

## Work-based Project with Questioning

### Knowledge

K2, K6, **K9**, K11

### Skills

**S1**, S4, S6, **S7**, S10,  
**S11**, S12, S16

### Behaviours

B2, B3

# Lesson Aim and Objectives

**Lesson Aim:** C# Fundamentals: Syntax, Variables & Program Structure

## Lesson Objectives

### Apply

- Apply C# syntax rules to construct simple executable programs

### Demonstrate

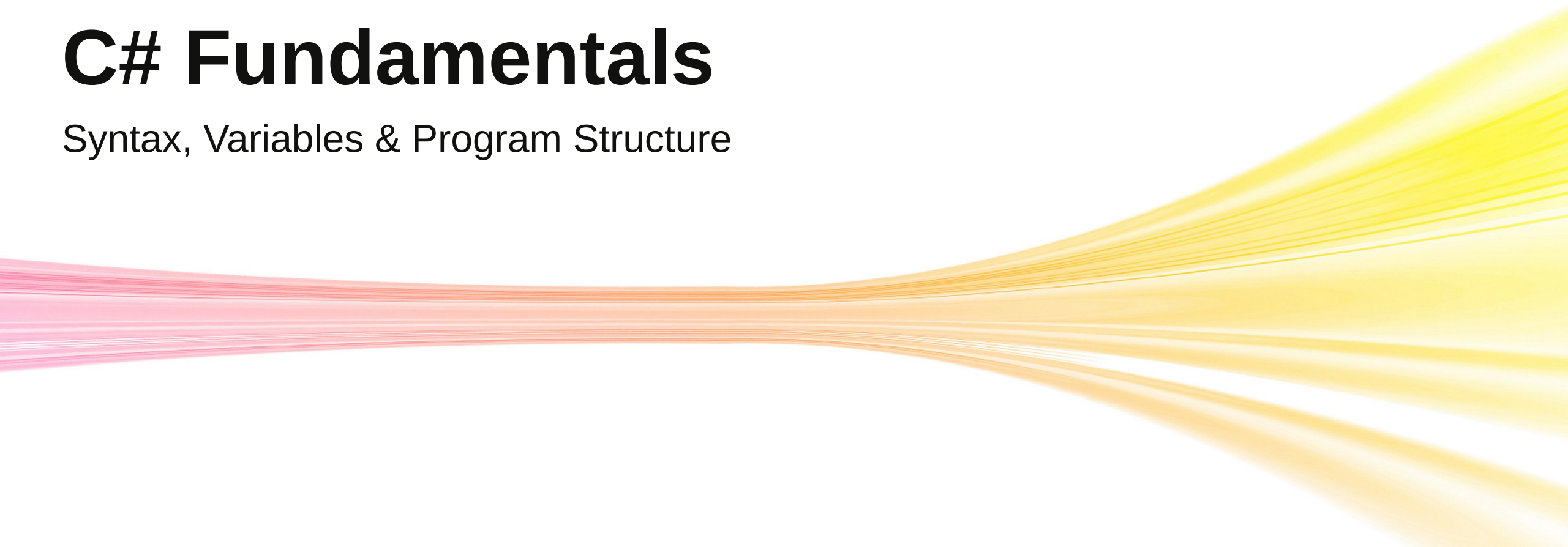
- Demonstrate use of variables and data types within a class-based structure

### Analyse

- Analyse program flow and identify syntax errors affecting execution

# C# Fundamentals

Syntax, Variables & Program Structure



# What is C#?



# Welcome to C#!

## How C# Executes Code

- C# is a compiled, strongly-typed language built around classes and methods
- Every program starts execution from the main method inside a class

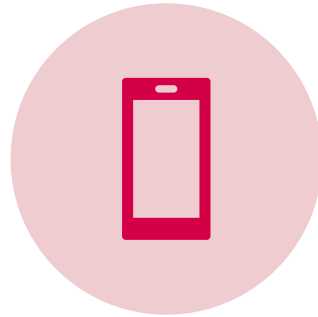
Example:

```
Console.WriteLine("Step 1: Preparing ingredients.");  
Console.WriteLine("Step 2: Cooking the meal.");  
Console.WriteLine("Step 3: Dinner is served!");
```

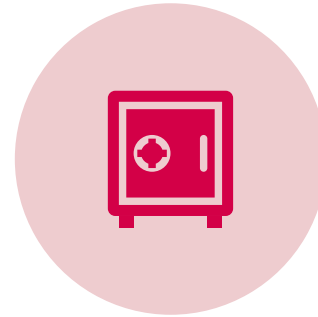
# Core Characteristics



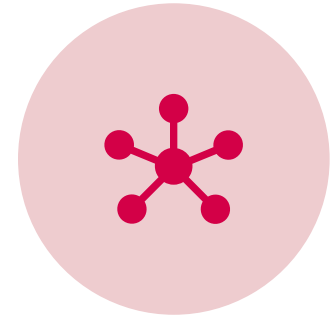
**COMPILED**



**STATICALLY  
TYPED**



**CROSS-  
PLATFORM (.NET)**



**RICH BASE  
CLASS LIBRARY**

# Core Syntax Rules

## Key Rules:

| Rule                   | Description                                                                                          |
|------------------------|------------------------------------------------------------------------------------------------------|
| <b>Braces</b>          | Curly braces { } group every block of code, unlike Python's indentation                              |
| <b>Statements</b>      | Semicolons ; terminate every statement, such as variable declarations and method calls               |
| <b>Parentheses</b>     | Round brackets () are used for calling methods like Console.WriteLine()                              |
| <b>Quotation Marks</b> | Strings must be wrapped in matching double "" quotes; single " quotes are reserved for a single char |

Consistency is key; a missing semicolon or brace will stop your code compiling

# Right vs. Wrong (Syntax Comparison)

## Correct Script:

```
if (true) {  
    Console.WriteLine("This is correct!"); }
```

## Flawed Script:

```
if true {  
    Console.WriteLine("This is incorrect!" }
```

What are the 3 errors in the flawed version?

- missing parentheses around the condition,
- missing semicolon,
- an unclosed parenthesis

## Corrected Script:

```
if (true){  
    Console.WriteLine("This is now correct!"); }
```

# Understanding Error Messages

Error messages are helpful clues, not failures

## Common Error Breakdown:

- **error: ';' expected** → You forgot a semicolon at the end of a statement
- **error: reached end of file while parsing** → You are missing a closing brace } somewhere in your code
- **error: 'Console' does not contain a definition for 'Writeline'** → You made a typo in a method name

**Tip:** Always fix the first error first — one missing brace or semicolon can cause many knock-on errors below it

# Anatomy of a C# Compiler Error

Deconstructing a Compiler Error:

```
Program.cs(3,34): error  
CS1002: ; expected  
    Console.WriteLine("Hello")  
                        ^  
1 error
```

## Key Components:

- **The File & Line:** Tells you exactly where the C# compiler got confused (Program.cs, line 3)
- **The Visual Indicator:** The caret symbol (^) points to where the error was detected
- **The Error Name:** Categorises the issue (e.g., ';' expected, CS0103 name not found, CS0029 cannot convert type, CS0162 unreachable code)
- **The Description:** Provides the specific detail (';' expected means a semicolon is missing at the end of a statement)

**Debugging Rule:** If line 3 looks perfectly fine, check line 2; a missing semicolon or brace above often causes the error reported below

# Variables and Data Types

- Variables are for **Storing** and **Updating** data, they're like containers that hold different kinds of information

## Key Data Types:

- **Integer (int)**: Whole numbers, like `int score = 25;`
- **Double (double)**: Decimal numbers, like `double price = 19.99;`
- **String (string)**: Text data, like `string city = "London";`
- **Boolean (bool)**: True/False values, like `bool isActive = true;`

**Updating Value:** You can change a variable's data later in the program (e.g., `score = score + 5;`)

^ What does this code do?

# Constants

- A **constant** is a type of variable whose value **cannot be changed** once it is set during the running of a program

## Why Use Constants?

- **Prevents Accidents:** It stops you or other programmers from accidentally overwriting critical data (like changing the value of Pi by mistake)
- **Easy Maintenance:** If a global value changes in the real world (like a VAT rate), you only have to update it in **one place** at the top of your code
- **Code Clarity:** In C#, constants are declared with the const keyword (e.g., `const double VAT_RATE = 0.20;`). Seeing `TOTAL * VAT_RATE` is much easier to understand than seeing `TOTAL * 0.20`

# Formatted Output with WriteLine()

## Mastering the WriteLine() Method

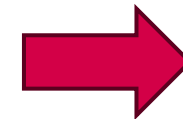
- Console.WriteLine() is used to display clear, formatted data to the user

## Key Techniques:

- **Basic Printing:** Passing text or numbers into Console.WriteLine()
- **String Concatenation:** Joining variables directly into strings using the + operator, e.g. "Text " + variable

## Code Example:

```
string name = "Alex";  
int score = 95;  
Console.WriteLine("User " + name + " scored " + score + " points!");
```



User Alex scored 95 points!

# Interactive Programs with Console.ReadLine()

- We can use Console.ReadLine() to **Capture** and **Process** user input
- It makes programs interactive by taking text from the keyboard

**The Golden Rule:** The Console.ReadLine() method ***always*** returns a **string** data type

**Type Conversion:** We can convert that string into a number using int.Parse() so you can do math

## Code Example:

```
Console.Write("Enter your age: ");  
string ageStr = Console.ReadLine();  
int age = int.Parse(ageStr); // Converts string to int  
Console.WriteLine("Next year you will be " + (age + 1));
```

Next year you will be 26

What will the output of this code look like if the user entered the number 25?

# Benefits of Structured Output Formatting

## 1. Superior Readability

- Using string interpolation (`$"..."`) keeps your text intact as a single template. Chaining many `+` operators chops your code into multiple fragments, making it harder to read at a glance

## 2. Automatic Data Type Conversion

- `Console.WriteLine()` and string interpolation automatically convert numbers, bools, and objects into readable text for you

## 3. Placeholder-Based Substitution

- Placeholders such as `{name}` or `{price:C}` let you drop values directly into a template string using `$"..."`, without manual concatenation

## 4. Built-in Text & Number Styling

- You can format numbers without altering the original data variables, allowing you to:
  - Round long decimals (e.g., limiting 3.14159 to just 3.14)
  - Format numbers as currency (e.g., adding commas to 1000000 to show 1,000,000)
  - Pad numbers with leading zeros (e.g., turning 5 into 05 for clock displays)
  - Align text into neat columns for terminal tables

# The Code-Run-Modify Cycle

- We use this method when **Experimenting** and **Observing** the behaviour of our script
- Developing code is an iterative process of trial and observation

## The Workflow:

- **Write:** Add or change a line of code in your script
- **Run:** Compile and execute the program to see the machine's response
- **Observe:** Look at how the output or errors change

**Best Practice:** Change only one thing at a time so you know exactly what caused a new behaviour



# **FIREBRAND**

**LYCEUM EDUCATION GROUP**

## **Demo**

CSharpDemo1.cs

# Demo step-by-step

- Open **CSharpDemo1.cs** from the Git Repo (it is recommended to use Visual Studio or VS Code)
- Run the code to make sure it functions properly

## Task 1:

- Try changing the data types (e.g., change itemPrice from a double to an int)
- Break the code intentionally (e.g., delete a semicolon or a brace) to practice reading the resulting compiler error

## Task 2: Add VAT

- Update the total price to include 20% VAT

## Task 3: Add a Discount

- Add a flag (Boolean) to determine whether a discount should be applied
- Deduct a flat £2.00 off the order total

# Daily Work Logger



## ACTIVITY



**Objective:** Build a simple time-tracking command-line application

**Core Core Requirements:**

- Capture user text inputs
- Store data in variables
- Display formatted summaries

**Key Challenge:** Ensure data types match your tracking needs

**Extension Activity:** Capture multiple entries and calculate total duration



## ACTIVITY



# Structured Debugging

We need to shift from random trial-and-error to a methodical debugging strategy

- **Why It Matters:**

Guessing changes wastes time; reading clues solves bugs permanently

- **The Mission:**

Analyse pre-written, broken code scripts and repair them systematically

- **Expected Outcome:**

Documented evidence of a logical troubleshooting approach.

# Structured Debugging Activity



## ACTIVITY



Open the file **Activity1.cs** from the Git Repo

**Objective:** Evidence a structured approach to debugging

**Task 1: Identify and Annotate:**

Find the bug and write a brief comment explaining the flaw

**Task 2: Correct and Re-run:**

Fix the specific problem line and test the file execution

**Task 3: Classify the Type:**

Label the mistake to understand exactly why it failed (e.g., syntax, runtime, logic)



## ACTIVITY



# Introduction to Code Quality

We want to evaluate scripts beyond whether they simply "work" or "run"

- **Why It Matters:**

- Code is read far more often than it is written

- Programs must be built for human developers to understand, not just for computers to run

- **The Mission:**

- Review, critique, and audit real-world code solutions

- Evaluate structural layouts, name clarity, and documentation habits

- **Expected Outcome:**

- Clearly articulate what makes an application clear and maintainable

- Write scripts that teammates can immediately understand and modify

# Code Quality Activity



Open the file **Activity2.cs** from the Git Repo

**Objective:** Articulate what makes code clear and maintainable

**Task 1: Identify and Annotate:**

Find the problem areas and annotate them using comments (e.g., using //)

**Task 2: Correct and Re-run:**

Re-write the code to make it more readable

**Task 3: Classify the Type:**

Label the mistake to understand exactly why it failed (e.g., syntax, runtime, logic)



ACTIVITY



# Stretch and Challenge

## The Mission:

Upgrade your working logger application into production-grade code

- **Task 1: Consistent Naming Conventions**

Convert every variable name to standard C# camelCase

Ensure zero single-letter variables remain in your script

- **Task 2: Structural Documentation**

Add meaningful comments explaining the underlying logic.

Use visual dividers to separate input, processing, and output sections.

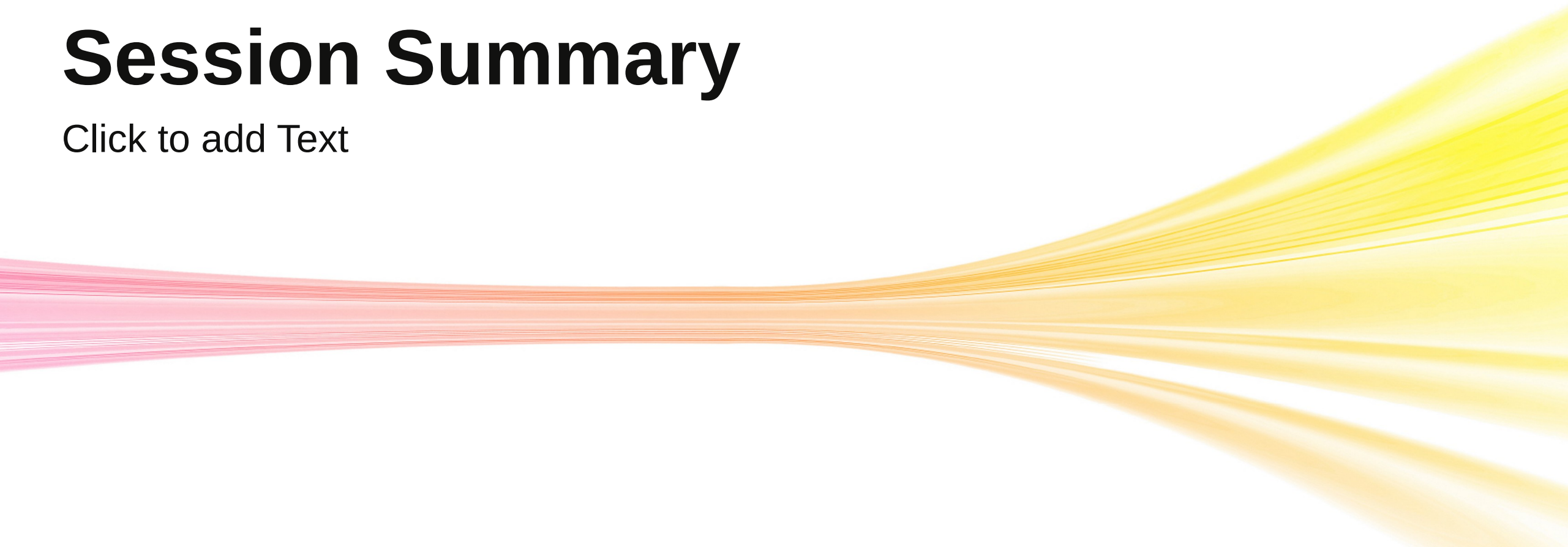
- **Task 3: Premium Output Formatting**

Redesign the terminal receipt using aligned spacing and structural borders.

Make the data instantly scannable for a manager or client review.

# Session Summary

Click to add Text



# Lesson Aim and Objectives

**Lesson Aim:** C# Fundamentals: Syntax, Variables & Program Structure

## Lesson Objectives

### Apply

- Apply C# syntax rules to construct simple executable programs

### Demonstrate

- Demonstrate use of variables and data types within a class-based structure

### Analyse

- Analyse program flow and identify syntax errors affecting execution

# Competencies and Coverage

## Professional Discussion underpinned by Portfolio

### Knowledge

**K1**, K3, K4, K5, K7, K8,  
K10, K12

### Skills

S2, S3, S5, S8, S9, S13,  
S14, S15, S17

### Behaviours

B1, B4, B5, B6, B7, B8,  
B9

## Work-based Project with Questioning

### Knowledge

K2, K6, **K9**, K11

### Skills

**S1**, S4, S6, **S7**, S10,  
**S11**, S12, S16

### Behaviours

B2, B3

